TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

3 créditos

Profesor:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Kenny Fabricio Moreira Mendoza

PARALELO: "A"

SEMESTRE: Quinto

ACTIVIDAD: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027

Actividades a desarrollar en la Unidad 1

Actividad # 1

INFORME TÉCNICO: SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL

1. Análisis del Problema y Requerimientos

1.1. Identificación de Actores

- **Ciudadano / Reportante:** Usuario externo que genera alertas sobre animales abandonados, heridos o en peligro.
- **Voluntario / Veterinario:** Personal operativo responsable de atender las alertas, evaluar la salud del animal, aplicar tratamientos y actualizar su estado.
- **Administrador del Refugio:** Gestiona el sistema, coordina asignaciones, aprueba solicitudes de adopción y genera reportes de gestión.

1.2. Requerimientos Funcionales (RF)

- **RF-01 (Registro de Alertas):** El ciudadano puede crear un caso de rescate con geolocalización, fotos y detalles del estado del animal.
- **RF-02 (Categorización de Expedientes):** El sistema debe generar expedientes técnicos específicos según la especie (*Perro*, *Gato*, *Fauna Silvestre*).
- **RF-03 (Asignación y Notificación):** Asignación de personal operativo a casos y envío automático de alertas cuando se ingresa un rescate en su zona.
- **RF-04 (Gestión de Estados):** Control de la traza de estados del animal (Rescatado $\rightarrow$ En Tratamiento $\rightarrow$ Disponible para Adopción $\rightarrow$ Adoptado).

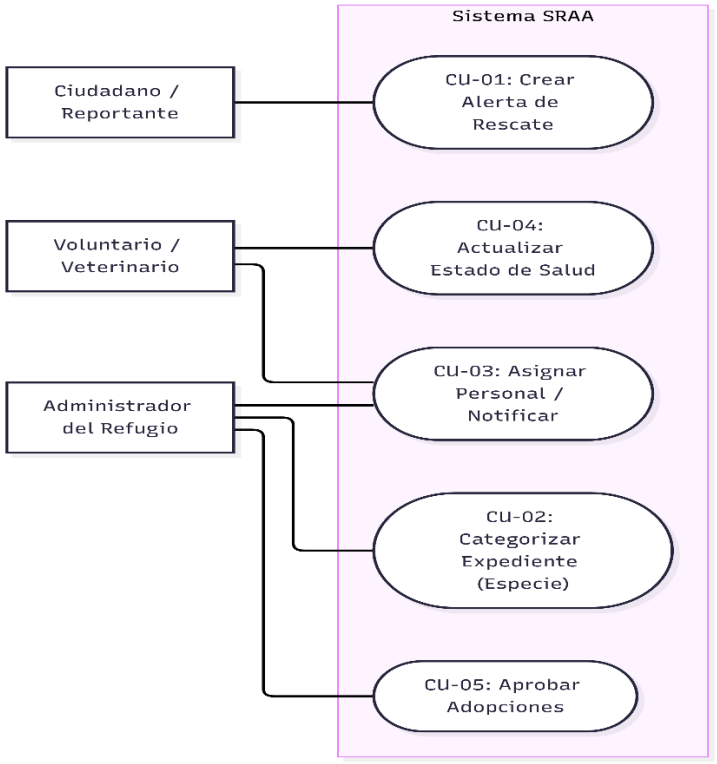


Figura 1. *Diagrama de Casos de Uso UML del Sistema SRAA*

1.3. Requerimientos No Funcionales (RNF)

- **RNF-01 (Escalabilidad):** Desacoplamiento de componentes para soportar la integración de nuevos tipos de animales o notificaciones.
- **RNF-02 (Mantenibilidad):** Aplicación de principios **SOLID** y arquitectura **MVC**.
- **RNF-03 (Trazabilidad y Control de Versiones):** Control del ciclo de vida del código mediante el modelo **Gitflow**.

2. Aplicación de Patrones de Diseño

Patrón	Tipo	Propósito y Justificación Técnica en el Sistema
Factory Method	Creacional	Instancia expedientes de rescate según la especie (ExpedientePerro, ExpedienteGato, ExpedienteFaunaSilvestre). Evita acoplar la lógica de rescates a clases concretas y facilita añadir especies a futuro sin modificar el código existente (Principio Open/Closed).
Observer	Comportamiento	Define una relación uno-a-muchos entre el sistema de ingresos y los voluntarios. Al registrar un rescate, el Sujeto (*RescateSubject*) notifica automáticamente a los Observadores (*VoluntarioObserver*, *VeterinarioObserver*) para coordinar la respuesta inmediata.
Singleton	Creacional	Garantiza que exista una única instancia global de la base de datos y del motor central del sistema (*ConexionBD* / *CasoManager*), evitando colisiones, consumo ineficiente de memoria y problemas de concurrencia.
MVC	Estructural	Separa los datos de dominio (*Model*), las vistas para el personal y ciudadanos (*View*), y los controladores (*Controller*) que procesan la lógica del negocio.

3. Arquitectura del Sistema y Diagrama de Clases UML

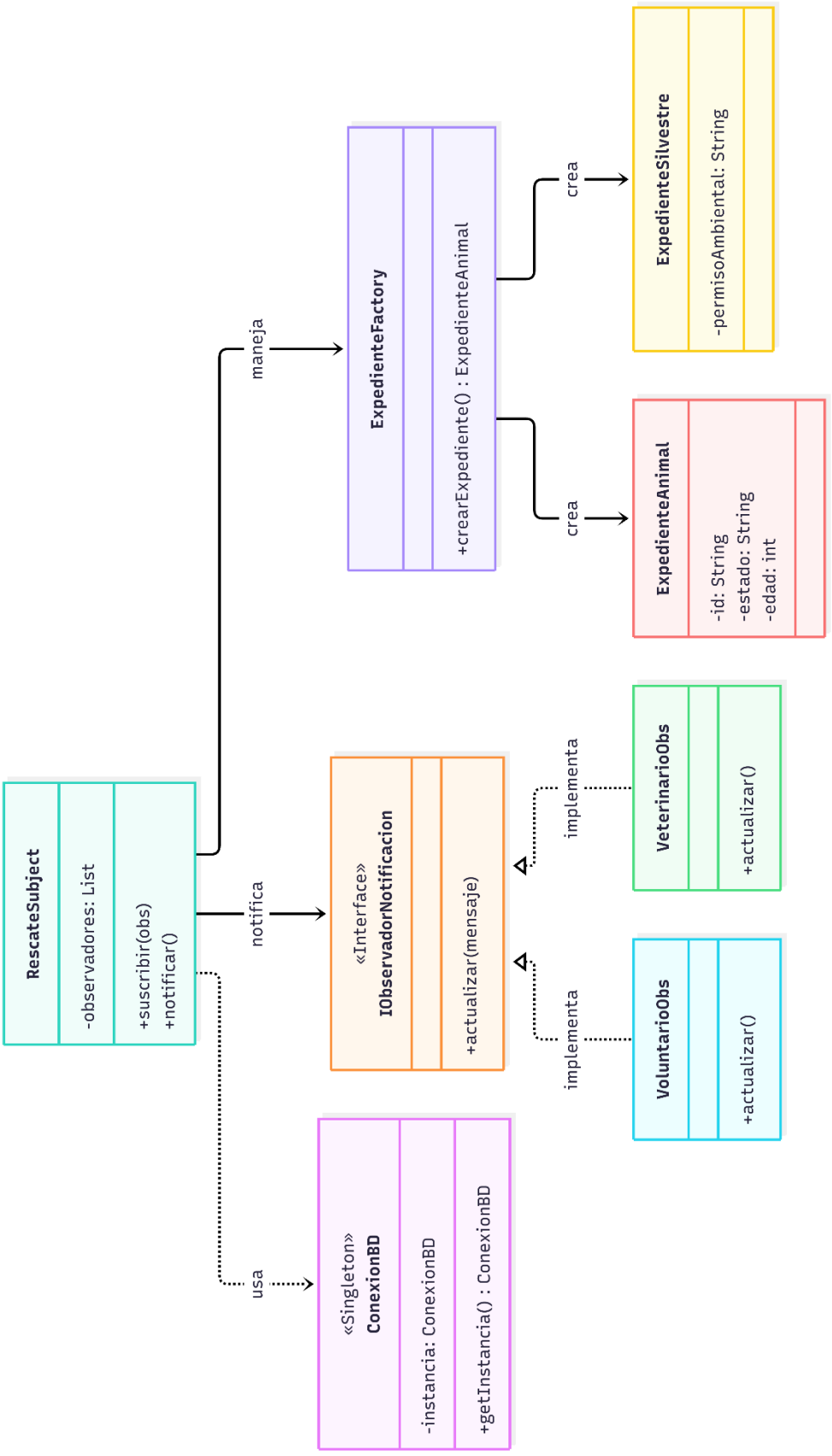


Figura 2. Diagrama de Clases (Modelado bajo Principios SOLID)

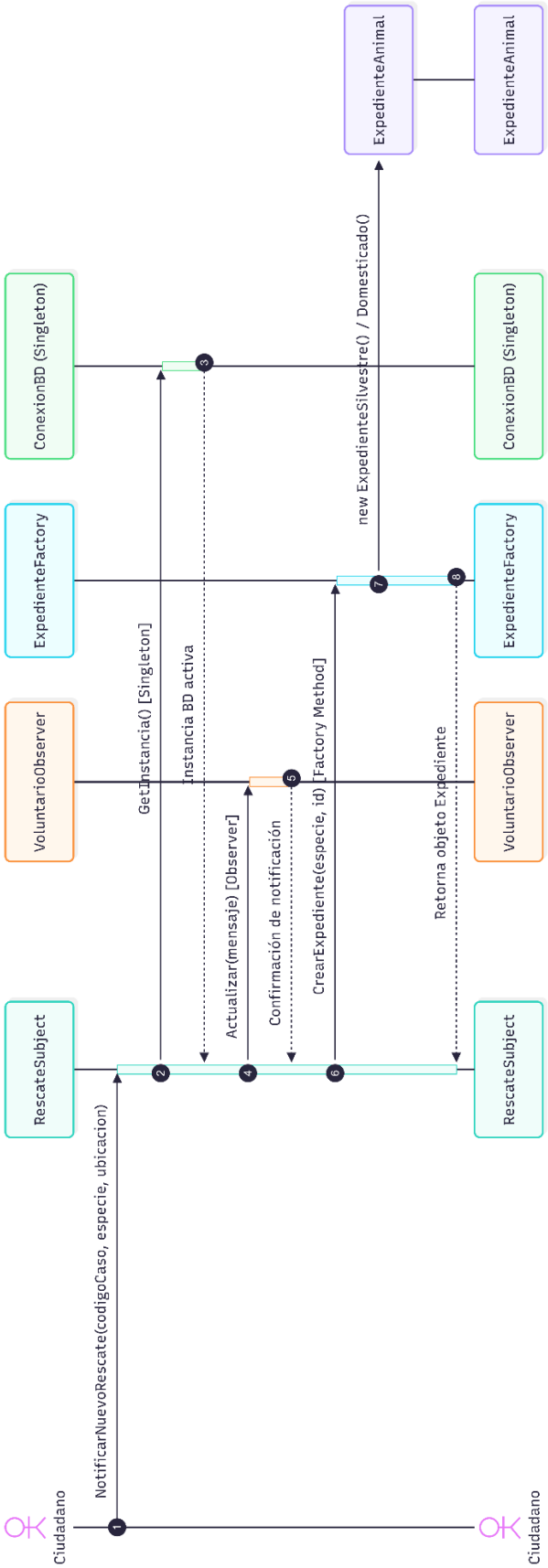


Figura 3. *Diagrama de Secuencia UML: Flujo dinámico de atención de un rescate e integración de los patrones Singleton, Observer y Factory Method.*

4. Estrategia de Control de Versiones: Modelo Gitflow

Para mantener un desarrollo ordenado y trazable en GitHub, el repositorio sigue la convención Gitflow:

- main / master: Código estable en producción listo para despliegue.
- develop: Rama principal de integración donde confluyen los avances probados.
- feature/ *(por ejemplo, feature/factory-expedientes, feature/notificaciones-observer)*: Ramas temporales creadas para desarrollar funcionalidades específicas que, tras su revisión, se integran en develop.
- release/: Preparación de versiones candidatas a producción.
- hotfix/: Correcciones críticas sobre la rama main.

4.1. Evidencia de Implementación y Repositorio Público

El desarrollo completo del sistema, la configuración del flujo de ramas y la integración de los patrones de diseño se encuentran alojados en el repositorio oficial del proyecto:

- **Enlace del Repositorio Público en GitHub:** https://github.com/Kenmomen1/SRAA-Sistema-Rescate-Animal2

Diagrama de Red de Ramas (Network Graph)

A continuación, se evidencia el gráfico de red de GitHub (Network Graph), donde se observa la creación independiente de las ramas feature/ a partir de develop y su posterior fusión unificada mediante Pull Requests:

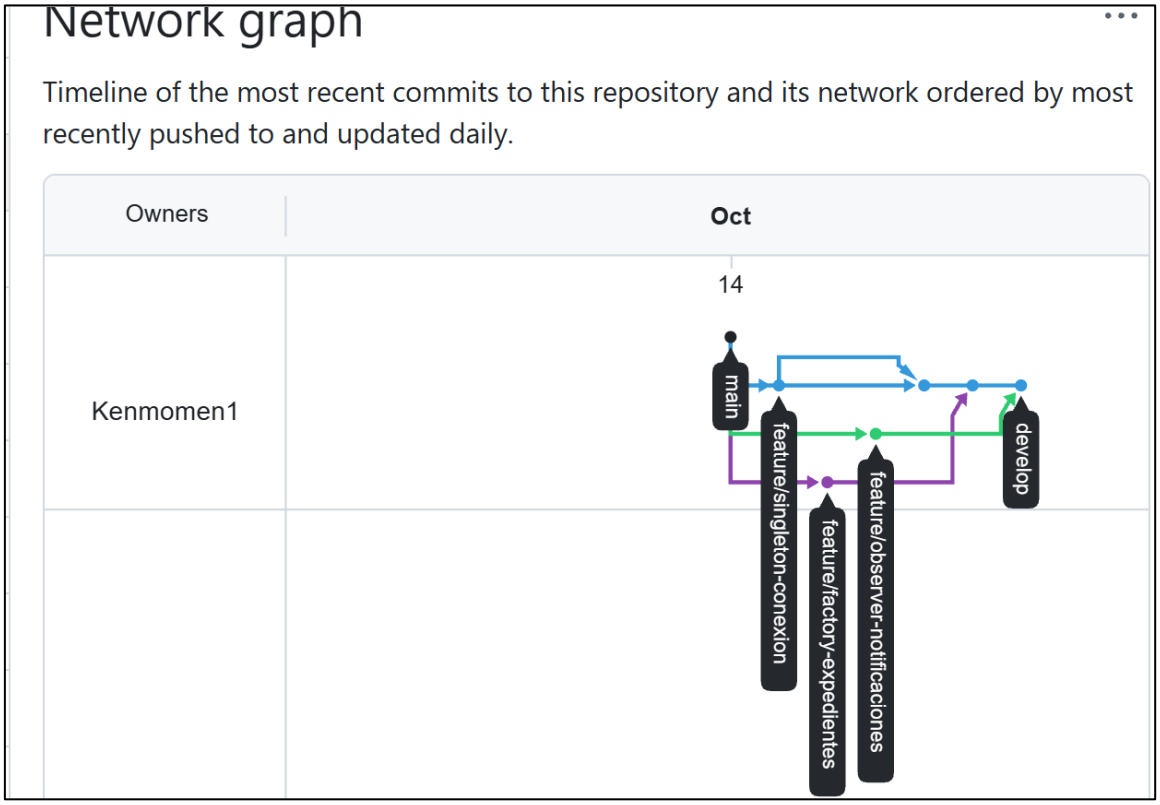


Figura 4. *Grafo de bifurcación y fusión de ramas bajo el modelo Gitflow.*

5. Código de Referencia para la Implementación de Patrones

A continuación, se muestra una estructura clara en C# / Java como base para los diagramas de secuencia e implementación del repositorio:

Código de Referencia C#

```
using System;
using System.Collections.Generic;

namespace SistemaRescateAnimal
{
    // ========================================================
    // 1. PATRÓN SINGLETON: Control de Conexión Única
    // ========================================================
    public class ConexionBD
    {
        private static ConexionBD _instancia;
        private static readonly object _lock = new object();

        private ConexionBD()
        {
            Console.WriteLine("[Singleton] Conexión a la Base de Datos inicializada con
éxito.");
        }

        public static ConexionBD GetInstancia()
        {
            lock (_lock)
            {
                if (_instancia == null)
                {
                    _instancia = new ConexionBD();
                }
                return _instancia;
            }
        }

        public void Conectar() => Console.WriteLine("[BD] Operación ejecutada sobre la
BD.");
    }

    // ========================================================
```

```
// 2. PATRÓN FACTORY METHOD: Creación de Expedientes
// ======================================================
public abstract class ExpedienteAnimal
{
    public string IdExpediente { get; set; }
    public string Especie { get; set; }
    public string EstadoSalud { get; set; }

    public abstract void MostrarResumen();
}

public class ExpedienteDomesticado : ExpedienteAnimal
{
    public bool TieneChip { get; set; }

    public override void MostrarResumen()
    {
        Console.WriteLine($"[Expediente Doméstico] ID: {IdExpediente} | Especie:
{Especie} | Chip: {TieneChip}");
    }
}

public class ExpedienteSilvestre : ExpedienteAnimal
{
    public string PermisoAmbiental { get; set; }

    public override void MostrarResumen()
    {
        Console.WriteLine($"[Expediente Silvestre] ID: {IdExpediente} | Especie:
{Especie} | Permiso MAE: {PermisoAmbiental}");
    }
}

public static class ExpedienteFactory
{
    public static ExpedienteAnimal CrearExpediente(string tipo, string id)
    {
        switch (tipo.ToLower())
        {
            case "perro":
```

```
            case "gato":
                return new ExpedienteDomesticado { IdExpediente = id, Especie = tipo,
TieneChip = false };
            case "silvestre":
                return new ExpedienteSilvestre { IdExpediente = id, Especie = "Fauna
Silvestre", PermisoAmbiental = "MAE-2026-88" };
            default:
                throw new ArgumentException("Especie no soportada para creación de
expediente.");
        }
    }
}

// ======================================================
// 3. PATRÓN OBSERVER: Notificaciones Reactivas
// ======================================================
public interface IObservadorNotificacion
{
    void Actualizar(string mensaje);
}

public class VoluntarioObserver : IObservadorNotificacion
{
    public string Nombre { get; set; }
    public VoluntarioObserver(string nombre) => Nombre = nombre;

    public void Actualizar(string mensaje)
    {
        Console.WriteLine($"[Notificación a Voluntario {Nombre}]: {mensaje}");
    }
}

public class VeterinarioObserver : IObservadorNotificacion
{
    public string Nombre { get; set; }
    public VeterinarioObserver(string nombre) => Nombre = nombre;

    public void Actualizar(string mensaje)
    {
        Console.WriteLine($"[ALERTA MÉDICA a Dr(a). {Nombre}]: {mensaje}");
```

```
        }
    }

    public class RescateSubject
    {
        private readonly List<IObservadorNotificacion> _observadores = new
List<IObservadorNotificacion>();

        public void Suscribir(IObservadorNotificacion obs) => _observadores.Add(obs);
        public void Desuscribir(IObservadorNotificacion obs) =>
_observadores.Remove(obs);

        public void NotificarNuevoRescate(string codigoCaso, string especie, string
ubicacion)
        {
            string mensaje = $"¡Nuevo rescate ingresado! Código: {codigoCaso} | Especie:
{especie} | Ubicación: {ubicacion}";
            foreach (var obs in _observadores)
            {
                obs.Actualizar(mensaje);
            }
        }
    }

    // ========================================================
    // PROGRAMA PRINCIPAL (Demostración de Integración)
    // ========================================================
    class Program
    {
        static void Main(string[] args)
        {
            // Instancia de Singleton
            ConexionBD bd = ConexionBD.GetInstancia();
            bd.Conectar();

            // Configurar Observers
            RescateSubject moduloRescates = new RescateSubject();
            moduloRescates.Suscribir(new VoluntarioObserver("Carlos Gómez"));
            moduloRescates.Suscribir(new VeterinarioObserver("Dra. Ana Martínez"));
```

```
        // Procesar un rescate y notificar
        moduloRescates.NotificarNuevoRescate("CASO-2026-001", "Fauna Silvestre",
"Sector Parque Central");

        // Crear expediente con Factory Method
        ExpedienteAnimal exp = ExpedienteFactory.CrearExpediente("silvestre", "EXP-
001");
        exp.MostrarResumen();
    }
  }
}
```

6. Conclusiones y Recomendaciones

- La arquitectura planteada logra un alto nivel de desacoplamiento gracias a la aplicación combinada de patrones creacionales (Factory Method, Singleton) y de comportamiento (Observer).
- El modelo Gitflow asegura un control riguroso de cambios en el equipo de desarrollo, reduciendo la probabilidad de conflictos de código en producción.
- Se recomienda extender el sistema añadiendo patrones adicionales como *State* para formalizar el cambio autómata de estados del animal durante su rehabilitación.